

实验报告

深圳技术大学

项 目	内 容
课程编号	HE00239
课程名称	自然语言处理
实验名称	实验1 — NLP开发与基础文本处理
实验类型	验证性、必做（2 学时，个人独立完成）
班 级	
指导教师	缪尧、张媛
报 告 人	
学 号	
合作者/组号	无（个人独立完成）
实验地点	A2-3XX
实验时间	2026 年 月 日
提交时间	

得 分	教师签名	批改日期

一、实验目的

1. 掌握 Python 环境检查、脚本运行与文件路径的基本操作，不把别人电脑的绝对路径写进代码；
2. 掌握 UTF-8、UTF-8-SIG、GB18030、UTF-16 四种中文文本编码的识别与读取方法；
3. 掌握 Unicode 规范化（NFKC）、空白处理、隐私字段掩码与基础词频统计；
4. 学会用中间输出和断言核验处理结果，让每一步都可观察、可复核；
5. 能够完整记录“错误现象—原因定位—修改—复测”的调试过程。

此外，本人给自己加了一条要求：报告中出现的每一个数字都必须由程序实际跑出来，不允许凭印象填写——本实验后半段的两个错误正是因为写死的期望值而产生的。

二、实验原理

2.1 多编码探测：试错式解码与 BOM

中文文本在磁盘上只是一串字节，编码信息并不随文本传递。用错误编码去解释同一串字节会抛出 `UnicodeDecodeError`，但“没报错”并不等于“解对了”：`utf-16` 会把任意偶数长度的字节强行两两配对，几乎总能“解码成功”，得到的却是乱码。因此本实验按固定顺序 `utf-8` → `utf-8-sig` → `gb18030` → `utf-16` 逐个尝试，并把每一次失败的编码与异常类型记入诊断列表。

顺序不能随意调整：若把 `utf-16` 提前，绝大多数文件都会变成“解错了却通过了”。

BOM 需要单独处理。带 BOM 的 UTF-8（即 UTF-8-SIG）其字节序列本身仍是合法 UTF-8，用 `utf-8` 解码不会报错，只是首字符残留一个 `U+FEFF`。所以解码成功后必须再检查首字符，命中则改用 `utf-8-sig` 重读。这类“解码成功但数据脏”的情况，比直接抛异常更危险。

2.2 Unicode 规范化 NFKC

NFKC（Compatibility Decomposition followed by Canonical Composition）先做兼容分解、再做规范合成，把全角字符折叠为半角。本实验在 C 组数据上实测得到：全角数字 30（`U+FF13/U+FF10`）→ 30，全角连字符 —（`U+FF0D`）→ -，全角冒号：（`U+FF1A`）→ :，全角逗号，（`U+FF0C`）→ ,；而「。」（`U+3002`）没有兼容分解，NFKC 后保持原样。

规范化属于有损操作，风险在于“清洗过头”。指导书明确指出不能用“只保留汉字”的正则清洗，否则 128/78 mmHg、5 mg/片 这类关键医学信息会被整体删除。所以本实验的规范化只做 NFKC 与空白折叠两步，不做任何删词操作。

2.3 隐私字段掩码

手机号正则 `(?<!\d)(1[3-9]\d)\d{4}(\d{4})(?!\\d)` 在前后加了负向断言，作用是防止从更长的数字串中间截取一段来掩码，造成误伤。邮箱正则保留前 2 个字符，便于人工核对字段归属而不泄露完整地址。实现上使用 `re.subn` 而不是 `re.sub`，因为替换次数本身就是需要被断言核验的量。

2.4 规则切词与“多字优先”

切词正则把多字否定短语排在单字规则之前：

代码 1 切词规则（多字短语必须优先匹配）

```
TOKEN_PATTERN = re.compile(
    r"患者|建议|进行|不伴|否认|不得|无|未|[A-Za-z]+\d+(?:\.\d+)?|[\u4e00-\u9fff]"
)
```

正则的候选分支按书写顺序尝试。如果把“不伴”放到单字规则之后，它会被拆成“不”+“伴”，否定语义随即丢失——这正是医疗文本里最危险的一类错误。

这里有一个容易踩的口径问题：指导书要求断言 token 中含有“未”，因此“未见”必须被拆成未 + 见。我一度想把“未见”作为整词加入多字优先规则，试算后发现那样 token 里就不再有“未”，指导书自己的断言反而会失败。因此保留指导书口径，并在代码注释里写明理由。

2.5 停用词过滤与否定保护

通用停用词表通常是面向新闻、社交语料训练的，并不包含医疗否定词语义。如果把“无”“未”“不伴”当作停用词删掉，“未见异常放电”会变成“见异常放电”，“无意识障碍”会变成“意识障碍”——前者是没有症状，后者是确诊有病，语义完全反转。因此加载停用词之后必须减去否定与约束保护词。

本实验下发的 stopwords.txt 共 8 个词（的、了、和、与、于、患者、建议、进行），本身不含否定词，所以保护逻辑在本次数据上不改变结果。但我仍然把它写成代码，因为这是一道必要的防御：换一份停用词表就会立刻翻车（详见 5.4 调试 2）。

2.6 词频统计与中间输出

collections.Counter 的 most_common(n) 按词频降序取前 n 项。为了让处理过程可复核，程序逐组打印：识别编码、编码失败记录、BOM 说明、规范化前后片段、切词结果、被删停用词、Top-10 词频，以及“原始→规范化→掩码→token”四阶段字符数。

2.7 断言与分支的区别

assert 用于检查“正常情况下必须成立”的内部条件，适合实验测试与调试；if 用于处理预期中可能发生的业务分支。需要注意 python -O 优化模式会跳过断言，因此断言不能代替正式的用户输入校验，更不能用于医疗安全判断。

三、实验仪器与编程环境

项 目	内 容
计算机	x64 架构, Windows 10 (10.0.19045)
Python	3.14.6, 解释器 C:\Users\Administrator\AppData\Local\Programs\Python\Python314\python.exe
编程工具	VS Code / PowerShell 命令行, 一条命令 python expl_starter.py 跑通全流程
第三方依赖	无。仅使用标准库 re、unicodedata、csv、collections、pathlib
分词库	未安装 jieba。指导书 3.6 节规定的 tokenize 为正则实现, 不依赖分词库; 采用零依赖方案可保证换一台电脑也能复现
结果导出	Word COM 自动化导出 DOCX 与 PDF

说明：本次实验全部代码不写死任何他人电脑的绝对路径，一律以 Path(__file__).resolve().parent 为基准定位 data/ 与 stopwords.txt。

四、实验内容

4.1 数据文件与个人任务

实验资源包目录为 实验一/ ├── expl_starter.py ├── stopwords.txt └── data/。
A—D 为必做，E—J 为选做，选做不替代必做。本人完成了全部 A—D 必做组，并额外完成 E—J 全部 6 个选做组。

组	性质	文件名	主题	个人任务
A	必做	input_A_utf8.txt	健康数值与单位	保留血压、剂量单位、否定/约束词和段落
B	必做	input_B_utf8sig.txt	健康教育与隐私字段	掩码手机号和邮箱，同时保留血糖数值与单位
C	必做	input_C_gbl8030.txt	康复训练与全角字符	全角数字/标点规范化，保留日期、时间与检查术语
D	必做	input_D_utf16.txt	检查记录与否定表达	停用词处理后仍保留 无/未/不伴 等否定表达
E	选做	optional_E_utf8.txt	公共卫生宣教与段落	合并多余空白但保留两个语义段落
F	选做	optional_F_gbl8030.txt	心电检查说明	规范全角日期时间，保留动态心电图术语
G	选做	optional_G_utf8sig.txt	睡眠健康建议	只统一正文英文大小写，保留组别和 SIM-G 编号
H	选做	optional_H_utf16.txt	视力健康教育	使用停用词表并解释变化，同时保留“未”
I	选做	optional_I_utf8.txt	用药安全科普	规范标点但不得删除剂量单位和“不得”
J	选做	optional_J_gbl8030.txt	老年健康管理	输出原始/规范化/掩码/token 阶段统计和编码错误记录

4.2 处理流程

原始字节 → ① 多编码探测（含 BOM 检测） → ② NFKC 规范化 + 空白折叠 → ③ 隐私掩码（含命中计数） → ④ 规则切词（多字优先） → ⑤ 停用词过滤（否定保护） → ⑥ 词频统计 → ⑦ 断言核验 → ⑧ 结果写入独立目录 outputs/

- 原始文件一律不被修改，处理结果全部写入独立目录 outputs/，符合实验步骤 1 的要求。
- 每组先跑断言再落盘：语义一旦丢失，程序在写盘之前就中止。

4.3 必做组 A—D 结果摘要

组	识别编码	尝试次数	原始→规范化→掩码 →token 字符数	手机/邮箱	停用词删除	专项断言
A	utf-8	1	95 → 94 → 94 → 57	0 / 0	与、患者、患者	数值/单位/否定词/段落均保留（通过）
B	utf-8-sig	1	112 → 111 → 108 → 60	1 / 1	与、患者	手机号与邮箱各命中 1 处；血糖 6.2 mmol/L 保留（通过）
C	gbl8030	3	96 → 95 → 95 → 65	0 / 0	与、患者	全角已转半角，2026-10-15 09:30 与“动态心电图”保留（通过）
D	utf-16	4	84 → 83 → 83 → 58	0 / 0	与、患者、患者、建议、进行	过滤停用词后 无/未/不伴 仍保留（通过）

```
python expl_starter.py — 环境自检与停用词表

【第 0 部分】Python 环境自检
=====

Python 版本 : 3.14.6
解释器路径 : C:\Users\Administrator\AppData\Local\Programs\Python\Python314\python.exe
操作系统 : Windows 10
当前目录 : D:\深技大课程学习\AI time\自然语言处理 实验一\实验一
脚本目录 : D:\深技大课程学习\AI time\自然语言处理 实验一\实验一
[OK] 版本、data/ 与 stopwords.txt 均就绪

=====

【第 1 部分】加载停用词表
=====

停用词表: 共 8 个 -> 过滤后 8 个
保护命中中: 无（本数据组的停用词表不含否定词，保护逻辑不改变结果）

=====
```

图 1 环境自检与停用词表加载（真实运行输出）

4.4 【选做】选做组 E—J 结果与必做组的差异

组	识别编码	尝试次数	原始→规范化→掩码→token	规范化变化	否定词留存	段落
E	utf-8	1	88 → 86 → 86 → 65	字符数改变、全角 4→0	—	是
F	gb18030	3	85 → 84 → 84 → 58	字符数改变、全角 19→0	未	否
G	utf-8-sig	1	91 → 90 → 90 → 52	字符数改变、全角 5→0	—	否
H	utf-16	4	78 → 77 → 77 → 59	字符数改变、全角 4→0	未	否
I	utf-8	1	82 → 81 → 81 → 61	字符数改变、全角 4→0	不得	否
J	gb18030	3	81 → 80 → 80 → 63	字符数改变、全角 3→0	未	否

与必做组相比，选做组新增了三类考察点：

- E 组考察段落结构不被空白折叠破坏——原文“预防呼吸道传染病 应注意通风”中两个连续空格被合并为一个，而两个语义段落之间的空行必须保留，所以 E 组字符数是 2（折叠 1 加上首尾 strip 的 1），其余 9 组都是 1。
- G 组考察大小写统一的“边界”：只允许改正文，标题组别【任务数据G】与编号 SIM-G 不能动。实现上按行号切分，仅对第 3 行起做小写化，并断言 sim-g 未出现、caffeine 已出现。这是必做组没有的——考的是“该改哪些、不该改哪些”的判断力。
- J 组要求显式输出四阶段统计与编码错误记录，把中间结果落盘以便复查，对应 outputs/阶段统计与编码诊断.csv。

五、代码（附注解）与结果分析

5.1 关键代码片段

代码 2 多编码探测与 BOM 处理（指导书 3.2 节，并按实验步骤 2 补充 BOM 检查）

```
def read_text_checked(path):
    """依次尝试四种编码，成功即返回，失败信息记入 failed 列表作为诊断证据。"""
    failed = []
    for encoding in ("utf-8", "utf-8-sig", "gb18030", "utf-16"):
        try:
            text = Path(path).read_text(encoding=encoding)
        except UnicodeError as error:
            failed.append({"encoding": encoding, "error": type(error).__name__})
            continue
        # utf-8 能解码但残留 BOM —— 说明文件其实是 UTF-8-SIG，改用 utf-8-sig 重读
        if text.startswith("\uffff"):
            text = Path(path).read_text(encoding="utf-8-sig")
        return text, "utf-8-sig", failed, "utf-8 可解码但含 BOM，已改用 utf-8-sig 重读"
    return text, encoding, failed, "无 BOM"
    raise UnicodeError(f"无法识别文件编码: {path}")
```

代码 3 Unicode 规范化与空白处理（指导书 3.3 节 / 上机步骤 3）

```
def normalize_text(text):
    """NFKC → 统一换行符 → 空白折叠 → 压缩多余空行 → 去首尾空白。"""
    text = unicodedata.normalize("NFKC", text) # 全角→半角、兼容字符折叠
    text = text.replace("\r\n", "\n").replace("\r", "\n") # 统一换行符
    text = re.sub(r"[\t\u3000 ]+", " ", text) # 连续空白→单空格
    text = re.sub(r"\n{3,}", "\n\n", text) # 保留段落，不合并段落
    return text.strip()
```

代码 4 隐私字段掩码（指导书 3.4 节）

```
PHONE_RE = re.compile(r"(?!\\d)([3-9]\\d{4})(\\d{4})(?!\\d)")
EMAIL_RE = re.compile(r"([A-Za-z0-9._%+-]{2})[A-Za-z0-9._%+-]*(@[A-Za-z0-9.-]+)")
def privacy_mask(text):
    """掩码手机号与邮箱，并返回替换次数供断言核验。"""
    text, mobile_n = PHONE_RE.subn(r"l****2", text)
    text, email_n = EMAIL_RE.subn(r"l****2", text)
    return text, {"mobile": mobile_n, "email": email_n}
```

代码 5 断言记录与 D 组专项断言

```
# 每条断言显式携带 group / kind / idx, 汇总时直接取字段,
# 不从标签字符串里反解组号 (第一版正是栽在这里, 见 5.4 调试 4)
def check(group, kind, idx, text, condition, message):
    ok = bool(condition)
    label = f"{kind}[{group}{idx}] {text}"
    ASSERT_LOG.append({"group": group, "kind": kind, "idx": idx,
        "label": label, "ok": ok, "message": message})
    print(f" [{'PASS' if ok else 'FAIL'}] [{label}]"
    assert ok, f"{label} 失败: {message}"
# D 组专项断言: 停用词过滤后否定表达必须还在
need = ("无", "未", "不伴")
missing = [x for x in need if x not in res["tokens"]]
check("D", "专项", 1, "过滤停用词后 否定词 无/未/不伴 仍保留",
not missing, f"D 组否定表达丢失: {missing}")
```

5.2 运行结果

```
python expl_starter.py -- 【B 组 · 必做】健康教育与隐私字段 文件: input_B_utf8sig.txt

【B 组 · 必做】健康教育与隐私字段 文件: input_B_utf8sig.txt
个人任务: 掩码手机号和邮箱, 同时保留血糖数值与单位

=====

识别编码      : utf-8-sig
编码失败记录  : 无 (首个候选即成功)
BOM 说明      : utf-8 可解码但首字符含 BOM, 已改用 utf-8-sig 重读
汉字占比(诊断): 46.43%
规范化前片段  : '空腹血糖为6.2 mmol/L。随访号码13912345671, 邮箱studentB@example.com; 本例均为合'
规范化后片段  : '空腹血糖为6.2 mmol/L。随访号码13912345671, 邮箱studentB@example.com; 本例均为合'
隐私掩码      : 手机号×1, 邮箱×1 -> '空腹血糖为6.2 mmol/L。随访号码139****5671, 邮箱st***@example.com; 本例均为合成数据'
切词结果      : ['任', '务', '数', '据', 'B', '健', '康', '教', '育', '与', '隐', '私', '字', '段', '编', '号', 'SIM', 'B', '以', '下', '内', '容', '为', '合', '成', '数', '据']
停用词删除    : ['与', '患者']
过滤后 Top-10 : [(C 为', 3), (C 数', 2), (C 据', 2), (C B', 2), (C 教', 2), (C 号', 2), (C 合', 2), (C 成', 2), (C 本', 2), (C 任', 1)]
阶段字符数    : 原始112 -> 规范化111 -> 掩码108 -> token60

--- 断言核验: B 组 (input_B_utf8sig.txt) ---
[PASS] 通用[B1] 解码无替换字符 U+FFFF
[PASS] 通用[B2] 掩码后无 11 位手机号明文
[PASS] 专项[B1] 手机号与邮箱各命中 1 处
[PASS] 专项[B2] 掩码后保留血糖数值与单位 6.2 mmol/L

=====
```

图 2 B 组 (UTF-8-SIG): BOM 改判、手机号与邮箱各掩码 1 处

```
python expl_starter.py -- 【C 组 · 必做】康复训练与全角字符 文件: input_C_gb18030.txt

【C 组 · 必做】康复训练与全角字符 文件: input_C_gb18030.txt
个人任务: 把全角数字/标点规范化, 并保留日期、时间和检查术语

=====

识别编码      : gb18030
编码失败记录  : [{"encoding": 'utf-8', 'error': 'UnicodeDecodeError'}, ('encoding': 'utf-8-sig', 'error': 'UnicodeDecodeError')]
BOM 说明      : 无 BOM
汉字占比(诊断): 61.46%
规范化前片段  : '康复训练: 上肢抬举 3 0 次。2 0 2 6 - 1 0 - 1 5 0 9: 3 0 完成动态心电图检查, 训练后未出现不适。'
规范化后片段  : '康复训练: 上肢抬举30次。2026-10-15 09:30完成动态心电图检查, 训练后未出现不适。'
切词结果      : ['任', '务', '数', '据', 'C', '康', '复', '训', '练', '与', '全', '角', '符', '号', 'SIM', 'C', '以', '下', '内', '容', '为', '合', '成', '数', '据']
停用词删除    : ['与', '患者']
过滤后 Top-10 : [(C 训', 3), (C 练', 3), (C C', 2), (C 康', 2), (C 复', 2), (C 成', 2), (C 不', 2), (C 30', 2), (C 任', 1), (C 务', 1)]
阶段字符数    : 原始96 -> 规范化95 -> 掩码95 -> token65

--- 断言核验: C 组 (input_C_gb18030.txt) ---
[PASS] 通用[C1] 解码无替换字符 U+FFFF
[PASS] 通用[C2] 掩码后无 11 位手机号明文
[PASS] 专项[C1] 全角已转半角且日期时间/检查术语保留

=====
```

图 3 C 组 (GB18030): 前两次编码探测失败, 全角字符转半角

```
python expl_starter.py -- 【D 组 · 必做】检查记录与否定表达 文件: input_D_utf16.txt

【D 组 · 必做】检查记录与否定表达 文件: input_D_utf16.txt
个人任务: 停用词处理后仍保留无、未、不伴等否定表达

=====

识别编码      : utf-16
编码失败记录  : [{"encoding": 'utf-8', 'error': 'UnicodeDecodeError'}, ('encoding': 'utf-8-sig', 'error': 'UnicodeDecodeError'), ('encoding': 'gb18030', 'error': 'UnicodeDecodeError')]
BOM 说明      : 无 BOM
汉字占比(诊断): 77.38%
规范化前片段  : '脑电图检查未见异常放电, 患者无意识障碍, 不作明显头痛。建议进行随访记录。'
规范化后片段  : '脑电图检查未见异常放电, 患者无意识障碍, 不作明显头痛。建议进行随访记录。'
切词结果      : ['任', '务', '数', '据', 'D', '检', '查', '记', '录', '与', '否', '定', '表', '编', '号', 'SIM', 'D', '以', '下', '内', '容', '为', '合', '成', '数', '据']
停用词删除    : ['与', '患者', '患者', '建议', '进行']
过滤后 Top-10 : [(C D', 2), (C 检', 2), (C 查', 2), (C 记', 2), (C 录', 2), (C 电', 2), (C 任', 1), (C 务', 1), (C 数', 1), (C 据', 1)]
阶段字符数    : 原始84 -> 规范化83 -> 掩码83 -> token58

--- 断言核验: D 组 (input_D_utf16.txt) ---
[PASS] 通用[D1] 解码无替换字符 U+FFFF
[PASS] 通用[D2] 掩码后无 11 位手机号明文
[PASS] 专项[D1] 过滤停用词后 否定词 无/未/不伴 仍保留

=====
```

图 4 D 组 (UTF-16): 四次探测才成功, 否定词未被停用词删除

```
python expl_starter.py — 断言汇总 (31 条全部通过)

【断言汇总】

=====

断言总数 31 条，通过 31 条，失败 0 条
含专项断言的数据组: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J']
A-D 每组专项断言条数: A=1, B=2, C=1, D=1

结果已写入: D:\深技大课程学习\AI time\自然语言处理 实验一\实验一\outputs

=====

实验一主流程完成: 10 组数据, 31 条断言全部通过。

=====
```

图 5 断言汇总：10 组数据共 31 条断言，全部通过

5.3 结果分析

(1) 编码识别的代价与证据价值。

- D 组 UTF-16 需要试满 4 次 (utf-8、utf-8-sig、gb18030 全部抛 UnicodeDecodeError)；C、F、J 组 GB18030 需要试 3 次；A、E、I 组首个候选即成功。失败记录不是废信息，它恰恰证明“为什么最后这个编码是对的”。
- 一个值得注意的陷阱：utf-16 几乎不会抛异常——它把任意偶数长度字节强行两两配对。对 E、I 两个 UTF-8 文件，utf-16 也能“解码成功”，但得到的是脍 饴 …这样的乱码。所以我额外打印了一个汉字占比作为可读性诊断指标（正式文本在 46% - 78% 之间，乱码远低于此），并用“解码后不含 U+FFFD”做硬断言。

(2) 规范化前后对比（实测）。

组	规范化前	规范化后	关键变化
C	康复训练：上肢抬举 3 0 次。2 0 2 6 — 1 0 — 1 5 0 9：3 0 完成动态心电图检查	康复训练：上肢抬举30次。2026-10-15 09:30完成动态心电图检查	全角数字、连字符、冒号、逗号全部转半角
A	患者无头痛，不伴胸痛。家庭血压 128/78 mmHg。	患者无头痛，不伴胸痛。家庭血压 128/78 mmHg。	全角逗号转半角；数值与单位原样保留
E	预防呼吸道传染病 应注意通风、手卫生。	预防呼吸道传染病 应注意通风、手卫生。	两个连续空格折叠为一个，段落空行保留

这里有一个必须自己发现的陷阱：字符数变化不能用来判断规范化是否生效。NFKC 的全角转半角是 1:1 映射，字符总数不变——A 组和 C 组在把 5 个和 22 个全角字符全部转成半角之后，字符数依然只比原文少 1（那 1 个来自首尾 strip()）。只有逐个比对码位才能确认规范化真的生效了。若只盯字符数，会得出“规范化没起作用”的错误结论。

(3) 隐私掩码的精确性。

B 组实测：手机号 13912345671 → 139****5671（命中 1 处，长度不变）；邮箱 studentB@example.com → st***@example.com（命中 1 处）。整条文本字符数 111 → 108，净减 3 全部来自邮箱变短，手机号掩码并不改变长度。同时血糖数值 6.2 mmol/L 完整保留——这正是 B 组专项断言要守住的东西：脱敏不能顺手把业务数据一起删掉。

(4) 停用词过滤的语义影响。

D 组切词后被删除的停用词为：与、患者、患者、建议、进行。删除后“无、未、不伴”三个否定表达全部存活。这里要注意口径——“未见异常放电”被切成 未+见 两个 token，但否定含义由“未”承担，因此并未丢失。反过来看，如果我把“未见”当成整词处理，token 里就不再有“未”，指导书第 6 条断言会先失败——这说明断言口径必须和切词口径对齐，而不是各写各的。

(5) 词频结果的解读。

A 组 Top-10 为：数×2、A×2、为×2、痛×2，其余均为 1 次。C 组为：训×3、练×3、C×2、康×2、复×2、成×2、不×2、30×2。这些结果符合预期——每篇文本仅 3-5 行、长度 78-112 字符，单字切分后词表高度稀疏，高频词多为标题与正文中的重复字。这说明规则切词在本实验的小样本上够用，但它产出的并不是真正的“词”：“训练”“康复”都被切成了单字，要得到可用的关键词必须引入词典或统计分词，这是实验二要解决的问题。

5.4 调试记录：报错现象—原因定位—修改—复测

实验过程中共记录 4 个真实错误，全部可用 debug_record.py 复现，修复均已并入主程序。

编号	报错现象	原因定位	修改	复测
调试 1	断言 AssertionError: B 组首行应以【任务数据B 开头，实际为 '\uffeff【任务数据B: 健康教育】	B 组是 UTF-8-SIG。带 BOM 的 UTF-8 字节序列仍是合法 UTF-8，所以 utf-8 解码不报错，但首字符残留 U+FEFF。属“成功解出脏数据”	解码成功后检查首字符是否为 BOM，命中则改用 utf-8-sig 重读	通过。识别编码由 utf-8 改判为 utf-8-sig，首行干净
调试 2	断言 AssertionError: D 组否定表达丢失: ['无', '未', '不作']	用一份“随手扩大的”停用词表（含无/未/不作）过滤，“未见异常放电”被清洗成“见异常放电”，语义直接反转	load_stopwords 加载后减去否定保护词 PROTECT_WORDS	通过。实际只删除 与/患者/建议/进行，否定词全部存活
调试 3	直接照抄指导书 3.5.1 节第 3 条样例断言，AssertionError: A 组语义信息丢失	样例断言写的是“5 mg/片”（带尾随空格），但原文“5 mg/片”后紧跟全角逗号 U+FF0C，根本不存在空格。指导书排版留白被 PDF 抽文本保留了下来	断言只写确定存在的子串“5 mg/片”，空格不入断言	通过。这说明断言的期望值必须来自实测数据，不能来自文档排版
调试 4	31 条数据断言全部 PASS，程序却崩在 AssertionError: A 组缺少专门断言	汇总处用 l.split("[")[1].split(" ")[0] 从标签文本反解组号，但通用标签是“通用[A]”、专项标签却是“专项[1]”，格式不一致导致匹配不到。真正的数据处理全部正确，崩溃发生在自检层	check() 改为显式接收 group/kind/idx 并存入字典，汇总直接取字段	通过。A 组专项断言条数统计为 1，不再误报

```
python debug_record.py -- 调试 1: BOM 陷阱

调试 1 UTF-8-SIG 的 BOM 陷阱 (文件: input_B_utf8sig.txt)
=====

[步骤 1] 按指导书原样实现 read_text_checked (候选 utf-8 在前)
识别编码 = utf-8, 失败记录 = []
首字符 = '\uffeff' 码位 = U+FEFF
首行 = '\uffeff【任务数据B: 健康教育与隐私字段】'

[步骤 2] 报错现象: 按'首行应以【开头'写断言
实际报错: AssertionError: B 组首行应以【任务数据B 开头，实际为 '\uffeff【任务数据B: 健康教育

[步骤 3] 原因定位
• 文件真实编码是 UTF-8-SIG, 前 3 字节为 BOM: ef bb bf
• 带 BOM 的 UTF-8 序列本身仍是合法 UTF-8, 所以 utf-8 解码**不会报错**, 但 U+FEFF 被当正文留下。
• 这是'成功解出脏数据', 比直接报错更危险。

[步骤 4] 修改: 解码成功后检查首字符是否为 BOM, 是则改用 utf-8-sig 重读
修改后识别编码 = utf-8-sig, 首行 = '【任务数据B: 健康教育与隐私字段】'
通过

[复测结论] 修改后首行干净, 断言通过。**此项已并入 expl_starter.py 的read_text_checked()。**

=====
```

图 6 调试 1 的实测记录 (BOM 陷阱)

其中调试 4 最让我反思。它是我自己写的“自检”把矛头指向了完全正确的数据处理结果：如果当时只看到报错信息，很容易以为 A 组漏写了断言，去反复检查本来没问题的处理逻辑。教训是——自检要断言数据本身，不要断言自己写的标签文本；能取字段就别解析字符串。

六、实验总结与感悟 (AI 辅助说明)

6.1 总结与感悟

最大的收获不是“会用 jieba 之外的切词”，而是理解了文本处理里最容易被忽略的一条原则：处理链路上任何一步都可能悄悄改变语义，而程序不会主动告诉你。

- “没报错”不等于“做对了”。utf-16 能把任意偶数长度字节解成“看似正常”的乱码；utf-8 能解开带 BOM 的文件却留下 U+FEFF。这两种情况都不会抛异常，只有主动检查（BOM 检测、替换字符检测、可读性诊断）才能拦住它们。
- 清洗的力度就是风险的来源。本实验如果图省事，用“只保留汉字”的正则一步到位，代码会更短，但 128/78 mmHg 和 5 mg/片 会全部消失——而在医疗语境里，删掉一个数值和写错一个数值，后果是同一量级的。
- 否定词是医疗文本的“地雷”。未见/无/不伴 参与构成句子极性，把它们扔进停用词表就像把句子里的“是”删掉。面向新闻训练的通用停用词表，直接套到医疗文本上是有缺陷的。
- 断言的价值在于把“我以为对”变成“程序验过对”。31 条断言全部通过之前，我的处理结果都只是看起来合理；通过之后才是可交付的。
- 本次实验我一共踩了 4 个坑，其中 3 个来自我自己的代码（自检逻辑写错、期望值写死、切词口径想当然），只有 1 个来自指导书样例。这让我意识到：读懂题面不等于理解题面，抄示例代码尤其要自己先验一遍。

6.2 AI 辅助说明

本次实验使用了 AI 辅助工具，按要求说明如下。

项 目	说 明
工具 / 模型	MiniMax Code（桌面版），模型 MiniMax-M3.1-Flash-Preview，使用日期 2026-10-09
使用目的	① 解读实验指导书与数据文件，梳理必做/选做要求；② 生成处理流水线初稿与断言；③ 辅助调试；④ 排版生成实验报告
关键提问概述	“依据参考资料完成实验一并制作报告”；“指导书 3.5.1 节的样例断言里 5 mg/片 后面为什么有空格”；“31 条断言都 PASS，程序为什么反而报 A 组缺少专门断言”
采用内容	多编码探测 + BOM 检查的整体思路、NFKC 规范化流程、正则掩码与 subn 计数、否定保护词机制、断言的分层组织、以及报告的排版框架
人工核验	所有数字均由本机实际运行产生，未采纳任何未经运行的推测值：31 条主流程断言、60 张任务卡共 286 条断言、各组字符数与词频均由 outputs/ 下的产物核对。特别地，AI 初稿曾直接沿用指导书样例断言中的“5 mg/片 ”（带尾随空格），本人在本机运行后发现该断言必然失败，逐一比对码位后改为“5 mg/片”；AI 给出的现场任务卡期望值也曾把“未见”当作整词，经检查与指导书断言口径冲突，已修正
收获与反思	AI 写代码很快，但它不会替你运行，也不会替你怀疑题面。本次 4 个错误里有 3 个是“看起来对”的代码——自检逻辑格式不一致、期望值硬编码、切词口径想当然。真正起作用的不是 AI 的产出，而是把它的产出当假设、逐条跑断言验证的习惯。报告里每一个数字都能追溯到 outputs/ 下的一个文件，这是我认为最有价值的部分

七、思考题

思考题 1 为什么不能把所有数字、英文和标点全部删除？结合个人数据举例。

因为这些字符恰恰承载了医疗文本最关键的信息。结合我的 A 组数据：“家庭血压 128/78 mmHg”“科普标签为5 mg/片”，如果把数字和英文全删，就只剩“家庭血压”“科普标签为片”——数值和剂量单位全部消失，这句话对医学没有任何意义。B 组的“空腹血糖为6.2 mmol/L”同理。英文部分也一样：G 组的 Sleep Health 和 CAFFEINE 是关键词。我的结论是，数字、单位、英文缩写属于“语义主体”，而标点才是真正可以清理的对象（NFKC 已经把全角

标点统一成半角，这一步就够了）。

思考题 2 errors="ignore" 可能造成什么问题？

它会让解码失败的部分被静默跳过，程序照常运行、照常输出，使用者完全看不到任何提示。在本实验里，我用 read_text_checked 时如果图省事加上 errors="ignore"，那么 C 组文件会被 utf-8 强行解码——所有汉字变成乱码，而程序不会报错。我会拿到一份“格式正确、词频正常、但内容全是乱码”的结果。更危险的是，这种错误会顺着流水线传下去：切词、词频、断言全都“正常”，只有最后人工抽查才发现问题。指导书明确要求不得使用，理由就在这里。相比之下，记录 failed 列表既保留了诊断证据，又不掩盖问题。

思考题 3 NFKC 规范化可能带来哪些收益和风险？

收益有三点：一是全角半角统一，让后续的正则不必同时写两套；二是检索和匹配变得可靠，“2 0 2 6”和“2026”视为同一个字符串；三是消除因输入法或来源系统不同而产生的字符差异。风险也有三点：其一是有害且不可逆，全角转半角之后无法还原；其二是它不只处理数字，也处理标点——我实测到「，」「：」「；」都被转成了半角，而中文习惯使用的「。」（U+3002）却保持不变，结果就是标点风格变得中西混杂；其三是 NFKC 会做兼容分解，某些看起来无关的字符可能被改写，例如 ① → (1)、m² → m2，这在需要保真原文的场景下是不能接受的。所以我的做法是：规范化只在“需要检索和统计”的环节使用，原始文本始终另存一份，绝不原地覆盖。

思考题 4 隐私正则可能产生哪些漏报或误报？

误报方面，手机号正则要求 1[3-9] 开头且前后不是数字，所以像 12345678901（1 开头）或 138123456789012（12 位）都不会被掩码——但这恰恰是真实存在的漏报。另外，如果一个人的手机号被写成“138-1234-5678”，正则就完全匹配不上。邮箱正则要求有 @ 和点号，写成“studentB[at]example[dot]com”这类反垃圾写法同样会漏掉。漏报还包括：微信、身份证号、银行卡、姓名+住址等本实验完全没有覆盖的字段。误报方面，规则本身写得比较保守，主要风险是 orderId 这类含数字的长串——我的正则靠前后数字断言规避了一部分，但仍不保证万无一失。结论是：正则脱敏只能作为第一道防线，本实验里的掩码也只是合成数据的演示，真实系统必须叠加词典、校验位、上下文规则，并配合人工复核。

电子签名		成绩评定	